

Tarea Semana 4: Segundo Análisis de Casos

Código Limpio, Datos de Texto y Validación

Objetivo

Resolver cuatro ejercicios fundamentales aplicando entrada de datos, limpieza de texto, conversión de tipos, condicionales y salidas claras con f-strings.

Cada ejercicio debe demostrar el razonamiento antes del código. No se trata solo de que el programa funcione: debe verse claramente qué datos entran, qué proceso se aplica y qué salida se espera.

Formato obligatorio para cada ejercicio

Complete las siguientes partes:

1. **Problema:** describa qué debe resolver el programa.
2. **Entrada:** indique qué datos solicita al usuario.
3. **Proceso:** explique la fórmula, condición o validación usada.
4. **Salida esperada:** describa qué mensaje debe mostrar.
5. **Código:** escriba y ejecute la solución.
6. **Pruebas:** documente tres escenarios:
 - Caso normal.
 - Caso límite.
 - Caso inválido.

Entrega

- Entregue este notebook `.ipynb` completo.
- Exporte también el trabajo a PDF.
- Como práctica opcional, entregue uno de los ejercicios adicionalmente como archivo `.py`.

Ejercicios obligatorios

#	Ejercicio	Descripción	Habilidades clave
1	Calculadora de propina inteligente	Solicita total de cuenta y porcentaje de propina. Calcula propina y total final.	float , operaciones, f-strings
2	Detector de número par/impar	Solicita un entero e indica si es par o impar.	int , %, if/else

#	Ejercicio	Descripción	Habilidades clave
3	Clasificador de edad	Solicita edad y clasifica como menor, adulto o adulto mayor.	int , if/elif/else
4	Validador de compra con descuento	Solicita cliente, monto y si es cliente frecuente. Aplica descuento según reglas.	strings, float , and/or , f-strings

Use `try-except` en los ejercicios donde el usuario deba ingresar números.

Problema #1: Calculadora de propina inteligente

Problema

El Problema a resolver es cuanta propina se va a dar y sumarlo al monto final, pidiendo al usuario los datos de cuanto es el total de la cuenta y cual es el porcentaje de la propina.

Entrada

La entrada en este caso sera de dos valores, la cuenta como flotante y el porcentaje de propina como flotante tambien.

Proceso

Evaluaremos que el monto de la cuenta no sea 0 o menor. Una vez hecho esto procederemos a validar si la propina es 0 o un numero positivo. Una vez las validaciones esten concluidas procederemos a sacar el valor numerico de la propina y sumarlo a la cuenta. Para eso usaremos la operacion de matematica diviendo entre 3 cuenta por propina dividido entre 100.

Salida esperada

La salida sera una cadena que indique el monto de la cuenta el monto de la propina y el total.

Pruebas manuales

Escenario	Datos de entrada	Resultado esperado	Resultado obtenido
Caso normal	Cuenta 1500 Propina 5%	Total 1575 Propina 75	Total 1575 Propina 75
Caso límite	Cuenta 1000000000 Propina 15%	Cuenta 1150000000.00. Propina 150000000.00	Cuenta 1150000000.00. Propina 150000000.00

Escenario	Datos de entrada	Resultado esperado	Resultado obtenido
Caso inválido	Cuenta -0	Por favor los montos digitados deben ser de numeros	Por favor los montos digitados deben ser de numeros

In []:

```

try:
    cuenta = float(input("Por Favor Digite el Monto de la Cuenta: ")) #Pedimo el mont
    propina = float(input("Por Favor Digite el Porcentaje que desea dejar de Propina

    if cuenta <= 0: #Verifico que La Cuenta no sea 0 o menos
        print("La Cuenta debe ser mayor a cero.")
    elif propina < 0: # Verifico que La propina no sea Negativa, el usuario puede des
        print("La propina no puede ser negativa.")
    else:
        total = cuenta + (cuenta * (propina / 100)) #A la cuenta le sumo el valor de
        print(f"El total de tu cuenta es: {total:.2f}.\nEl Subtotal sería: {cuenta:.2
except ValueError:
    print("Por favor los montos digitados deben ser de numeros.")

```

El total de tu cuenta es: 1500.00.

El Subtotal sería: 1500.00 y el monto de la propina: 0.00

Problema #2: Detector de número par/impar

Problema

El Usuario necesita identificar si el numero es par o impar. Se le solicitara un numero al usuario y el sistema debera devolver si es par o impar.

Entrada

En esta caso la entrada sera el numero a evaluar que debe ser un numero.

Proceso

En este caso evaluaremos si el numero ingresado por el usuario es par o impar. Para hacer esto usaremos el operador % de Python que nos dira si el resto de la division por el numero que digitemos en el codigo a continuacion siendo el resultado 1 o 0.

Salida esperada

La salida esperada en este caso es una cadena de string que nos diga si el numero digitado es par o impar.

Pruebas manuales

Escenario	Datos de entrada	Resultado esperado	Resultado obtenido
Caso normal	8	par	par
Caso límite	89742159	impar	impar
Caso inválido	89.5	Por favor digitar solamente numeros enteros	Por favor digitar solamente numeros enteros

In []:

```
try:
    numero = int(input("Por Favor Digite el numero a evaluar: ")) #Pedimos el numero
    resto = numero % 2 # Asigo el resto de la division entre dos a la variable resto
    print(resto)
    if resto: #Verifico si resto es 1 o Verdadero, en este caso es impar
        print(f"El numero: {numero}, es un numero impar.")
    else: #Si en la validacion anterior es falsa el valor es 0 o negativo siendo el n
        print(f"El numero: {numero}, es un numero par.")
except ValueError:
    print("Por favor digitar solamente numeros enteros.")
```

Problema #3: Clasificador de edad

Problema

Recibe la edad de la persona y clasifica en Menor menos de 18 años, Adulto de 18 a 64 y Adulto Mayor, mayor de 65

Entrada

En este caso la edad de la persona sera el dato de entrada.

Proceso

En este caso vamos a tomar el valor de entrada("Edad") y asignarlo a una variable llamada edad. Con esta variable realizaremos varias comparaciones para ver si la edad es mayor a 0, si la edad es mayor que 0 y menor de 18 cayendo en la categoria de Menor de Edad. Si por el contrario el valor de la edad esta en el rango de 18 a menos de 65 la edad sera Mayor de Edad. Ahora si la edad es mayor a 65 la persona es considerada Adulto Mayor.

Salida esperada

La salida esperada es una cadena de texto indicando que segun la edad digitada indicando la edad eso se hace menor de edad, mayor de edad o adulto mayor segun sea el caso.

Pruebas manuales

Escenario	Datos de entrada	Resultado esperado	Resultado obtenido
Caso normal	20	Mayor de edad	Mayor de edad
Caso límite	99	Adulto Mayor	Adulto Mayor
Caso inválido	trece	La edad debe ser digitada en numeros positivos	La edad debe ser digitada en numeros positivos

In []:

```
try:
    edad = int(input("Por Favor Digite su Edad : ")) #Pedimos La edad de La Persona
    if edad <= 0: #Verifico que La Edad no sea 0 o menos
        print(f"La edad digitada : {edad} Recuerda que la edad debe ser mayor a 0")
    elif edad > 0 and edad < 18: # Verifico que La edad sea entre 0 o mayor y menor 18
        print(f"Su edad es: {edad} eso te hace Menor de edad ")
    elif edad >= 18 and edad < 65: # Verifico que La edad sea entre 18 o mayor y menor 65
        print(f"Su edad es: {edad} eso te hace Mayor de edad ")
    else: # VAqui no se verifica La edad que que si no cubrio las condiciones anteriores
        print(f"Su edad es: {edad} eso te hace Adulto Mayor de edad ")
except ValueError:
    print("La edad debe ser digitada en numeros positivos.")
```

Problema #4: Validador de compra con descuento

Problema

Se debe validar la compra del cliente y tener en cuenta las reglas de Descuento. Si el cliente no es frecuente no tiene descuento. Si el cliente es frecuente se le aplicara un descuento dependiendo del monto de la compra: Entre 0 y 1000 5% Entre 1000 y 5000 10% Entre 5000 y 10000 15% Mas de 10000 20%

Entrada

Se le solicitara al cliente, el monto de la Compra y si es cliente frecuente o no.

Proceso

En este proceso vamos a evaluar primeramente si el cliente es frecuente o no. Una vez realizada dicha validacion procederemos a calcular el monto del descuento dependiendo del valor de la compra. Para esto utilizaremos comparaciones simples. Si el cliente es frecuente se le aplicara un descuento dependiendo del monto de la compra: Entre 0 y 1000 5%, Entre 1000 y 5000 10%, Entre 5000 y 10000 15%, Mas de 10000 20%.

Salida esperada

La salida esperada sera una cadena de texto con el monto de la compra, el monto de la factura y el descuento aplicado.

Pruebas manuales

Escenario	Datos de entrada	Resultado esperado	Resultado obtenido
Caso normal	JuAn 8000.00 si	Estimado : Juan el monto de su compra es: 8000.00, con un descuento de 1200.0, su factura es: 6800.00	Estimado : Juan el monto de su compra es: 8000.00, con un descuento de 1200.0, su factura es: 6800.00
Caso límite	Ronald	125000	no
Caso inválido	Julio	Trece mil	El monto debe ser digitado en numeros

In [3]:

```
try:
    cliente = input("Por Favor Digite su Nombre : ").lower().strip().capitalize()#Ped
    monto = float(input("Digite el monto de la Compra: "))
    frecuente = input("Digite si, si el cliente es frecuente o no, si no lo es: ").lo
    descuento = 0

    if frecuente == "si": #Verifico si el cliente es frecuente o no y aplico descuen
        if monto > 0 and monto <= 1000:
            descuento = 0.05
        elif monto <= 5000:
            descuento = 0.1
        elif monto < 10000:
            descuento = 0.15
        elif monto >= 10000:
            descuento = 0.20

    total = (monto - float((monto * descuento)))
    print(f"Estimado : {cliente} el monto de su compra es: {monto:.2f}, con un de
else:
    print(f"Estimado : {cliente} el monto de su cuenta es: {monto:.2f}\nGracias p
```

```
except ValueError:  
    print("El monto debe ser digitado en numeros")
```

Estimado : Juan el monto de su compra es: 8000.00, con un descuento de 1200.0, su factura es: 6800.00
Gracias por Preferirnos

Conclusión

Redacte una reflexión breve:

- ¿Qué validación le resultó más importante? Creo que todas las validaciones son importantes dependiendo mucho del escenario presentado. El código limpio y ordenado fácil de leer y la información presentada al usuario de manera simple, formateada y sencilla hacen que sea más fácil de usar el sistema y que el usuario cometa menos errores.
- ¿Qué error encontró durante las pruebas? Los errores más comunes de momento son los de lógica, las funciones hacen su trabajo pero la lógica necesita práctica para mejorar.
- ¿Cómo mejoró la limpieza del código la calidad del programa? Mucho, es más fácil de entender, leer, seguir y debuggear.